

Day 31. Recommender systems implementation detail

Mean normalization

Users who have not rated any movies

Movie	Alice(1)	Bob (2)	Carol (3)	Dave (4)	Eve (5)
Love at last	5	5	0	0	?
Romance forever	5	?	?	0	?
Cute puppies of love	?	4	0	?	?
Nonstop car chases	0	0	5	4	?
Swords vs. karate	0	0	5	?	?

$$\min_{\substack{w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \\ x^{(1)}, \dots, x^{(n_m)}}} \frac{1}{2} \sum_{(i,j): r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Mean Normalization

$$\begin{aligned} & \begin{bmatrix} 5 & 5 & 0 & 0 & ? \\ 5 & ? & ? & 0 & ? \\ ? & 4 & 0 & ? & ? \\ 0 & 0 & 5 & 4 & ? \\ 0 & 0 & 5 & 0 & ? \end{bmatrix} \begin{matrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{matrix} \\ & \mu = \begin{bmatrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{bmatrix} \quad \begin{bmatrix} 2.5 & 2.5 & -2.5 & -2.5 & ? \\ 2.5 & ? & ? & -2.5 & ? \\ ? & 2 & -2 & ? & ? \\ -2.25 & -2.25 & 2.75 & 1.75 & ? \\ -1.25 & -1.25 & 3.75 & -1.25 & ? \end{bmatrix} \end{aligned}$$

For user j , on movie i predict:

$$w^{(j)} \cdot x^{(i)} + b^{(j)} + \mu_i$$

$$y^{(i,j)} \downarrow \\ w^{(j)} \cdot b^{(j)} \cdot x^{(i)}$$

User 5 (Eve):

$$w^{(5)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad b^{(5)} = 0 \quad \underbrace{w^{(5)} \cdot x^{(1)} + b^{(5)}}_0 + \mu_1 = 2.5$$

moreover, compare to normalize on columns (users), normalizing on the movies might be more helpful in certain circumstances in real life.

TensorFlow implementation of collaborative filtering

tensorflow is helpful at automatically finding out the derivative of the cost function.

Derivatives in ML

Gradient descent algorithm

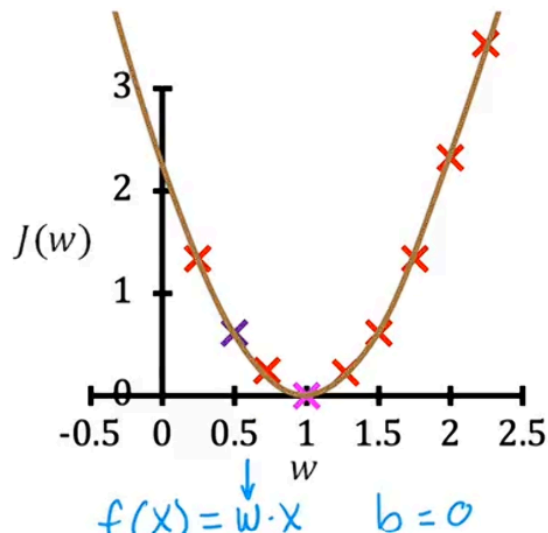
Repeat until convergence

$$\underline{w} = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

Learning rate

Derivative

$$\underline{b} = b - \alpha \frac{\partial}{\partial b} J(w, b) \leftarrow b = 0$$



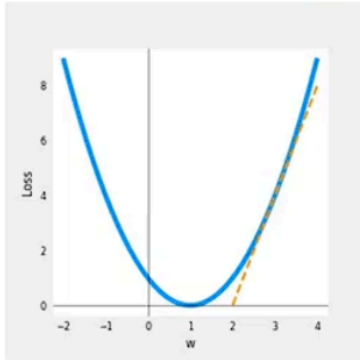
$$J = (w^{\overbrace{f(x)}} - \overbrace{y})^2$$

Gradient descent algorithm

Repeat until convergence

$$w = w - \alpha \frac{d}{dw} J(w, b)$$

Fix b = 0 for this example



Custom Training Loop

```
w = tf.Variable(3.0)
x = 1.0
y = 1.0 # target value
alpha = 0.01
```

Tf.variables are the parameters we want to optimize

```
iterations = 30
for iter in range(iterations):
    # Use TensorFlow's GradientTape to record the steps
    # used to compute the cost J, to enable auto differentiation.
    with tf.GradientTape() as tape:
        fwb = w*x
        costJ = (fwb - y)**2

    # Use the gradient tape to calculate the gradients
    # of the cost with respect to the parameter w.
    [dJdw] = tape.gradient(costJ, [w])

    # Run one step of gradient descent by updating
    # the value of w to reduce the cost.
    w.assign_add(-alpha * dJdw)
```

tf.variables require special function to modify

Implementation in TensorFlow

Gradient descent algorithm

Repeat until convergence

$$w = w - \alpha \frac{d}{dw} J(w, b, x)$$

$$b = b - \alpha \frac{d}{db} J(w, b, x)$$

$$x = x - \alpha \frac{d}{dx} J(w, b, x)$$

```
# Instantiate an optimizer.
optimizer = keras.optimizers.Adam(learning_rate=1e-1)

iterations = 200
for iter in range(iterations):
    # Use TensorFlow's GradientTape
    # to record the operations used to compute the cost
    with tf.GradientTape() as tape:
        # Compute the cost (forward pass is included in cost)
        cost_value = cofiCostFuncV(X, W, b, Ynorm, R,
                                   num_users, num_movies, lambda)

    # Use the gradient tape to automatically retrieve
    # the gradients of the trainable variables with respect to
    # the loss
    grads = tape.gradient(cost_value, [X, W, b])

    # Run one step of gradient descent by updating
    # the value of the variables to minimize the loss.
    optimizer.apply_gradients(zip(grads, [X, W, b]))
```

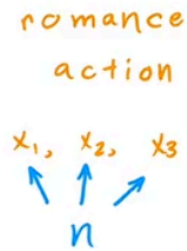
Dataset credit: Harper and Konstan. 2015. The MovieLens Datasets: History and Context

Finding related items

Finding related items

The features $x^{(i)}$ of item i are quite hard to interpret.
To find other items related to it,

find item k with $x^{(k)}$ similar to $x^{(i)}$



i.e. with smallest distance

$$\sum_{l=1}^n (x_l^{(k)} - x_l^{(i)})^2$$
$$\|x^{(k)} - x^{(i)}\|^2$$

Limitations of Collaborative Filtering

Cold start problem. How to

- rank new items that few users have rated?
- show something reasonable to new users who have rated few items?

Use side information about items or users:

- Item: Genre, movie stars, studio,
- User: Demographics (age, gender, location), expressed preferences, ...